Understanding Gradient Descent

Great question! Here's a clear walkthrough of how gradient descent actually works 【12†source】.

The core idea: we want to minimise a loss function (L(\theta)) by repeatedly stepping in the direction that reduces it fastest — the negative gradient. Each update looks like:

[\theta_{t+1} = \theta_t - \eta , \nabla_\theta L(\theta_t)]

where (\eta) is the **learning rate**.

A minimal implementation

```
import numpy as np

def gradient_descent(grad, theta, lr=0.1, steps=100):
    for _ in range(steps):
        theta = theta - lr * grad(theta)
    return theta
```

Copy code

Choosing the learning rate 【3†source】

- Too **small** → training crawls and may never converge in reasonable time
- Too **large** → the loss oscillates or diverges entirely :contentReference[oaicite:0]
- A good default is to start around `1e-3` and tune from there

Variants worth knowing

Variant	What changes	Good for
SGD	one sample per step	large datasets
Momentum	accumulates past gradients	ravines / plateaus
Adam	per-parameter adaptive rates	most deep-learning defaults